SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I **B Ragava Prasath / 192421026** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Q1. ATM Withdrawal Module – Reverse Engineering the Black-Box Test Design

1. Understanding of the system

Only the input-output behaviour of the withdrawal module is known. The observed log below is illustrative; the balance is the account balance before the request.

Obs	Amount (Rs.)	Balance (Rs.)	Observed output
O1	500	10,000	Success, new balance 9,500
O2	20,000	50,000	Success, new balance 30,000
O3	25,000	50,000	Rejected: "Exceeds per-transaction limit"
O4	550	10,000	Rejected: "Enter amount in multiples of 100"
O5	15,000	10,000	Rejected: "Insufficient balance"
O6	0	10,000	Rejected: "Invalid amount"
O7	abc	10,000	Rejected: "Invalid amount"

2. Decomposition into functional components

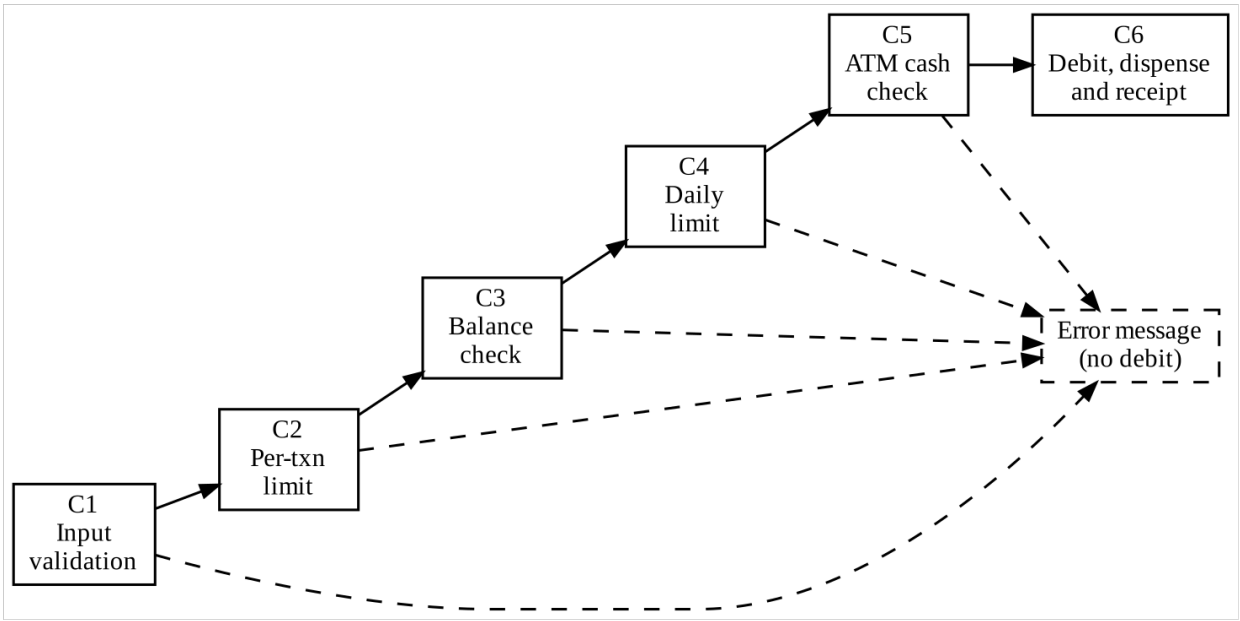


Fig. 1: Functional decomposition of the withdrawal module

Component	Function	Rule inferred from the behaviour	Evidence
C1 Input validation	Numeric, positive, correct denomination	amount > 0 and amount mod 100 = 0	O4, O6, O7
C2 Per-transaction limit	Upper limit of one withdrawal	100 ≤ amount ≤ 20,000 (20,000 accepted, 25,000 rejected; exact limit lies in 20,000 to 24,999)	O2, O3
C3 Balance validation	Funds available	amount ≤ balance (the equal case was never observed)	O5
C4 Daily limit	Cumulative limit per day	Not observable in the log (assumed Rs. 50,000 per day)	none
C5 ATM cash check	Cash available in the dispenser	Not observable in the log	none

Component	Function	Rule inferred from the behaviour	Evidence
C6 Debit and receipt	Update balance, dispense	new balance = balance − amount	O1, O2

3. Equivalence classes and boundary conditions

Class	Condition	Type	Covered by the observations?
EC1	amount ≤ 0	Invalid	Yes (O6)
EC2	Non-numeric or blank	Invalid	Yes (O7)
EC3	0 < amount < 100	Invalid	No
EC4	100 to 20,000 but not a multiple of 100	Invalid	Yes (O4)
EC5	100 to 20,000, multiple of 100, ≤ balance	Valid	Yes (O1, O2)
EC6	amount > 20,000	Invalid	Yes (O3)
EC7	Valid amount greater than the balance	Invalid	Yes (O5)
EC8	Valid amount but daily total above 50,000	Invalid	No

Boundary conditions: lower limit 100 (99, 100, 200), upper limit 20,000 (19,900, 20,000, 20,100), amount equal to the balance (balance, balance + 100) and the daily total (50,000 and 50,100). This gives 10 boundary test points.

4. Inferred black-box techniques and their adequacy

Technique inferred	Evidence	Adequacy
Equivalence partitioning (partial)	One test in each of 6 of the 8 classes (EC3 and EC8 are missing)	6 of 8 classes = 75%
Boundary value analysis (very weak)	Only the value 20,000 lies on a boundary; 99, 100, 200, 19,900, 20,100, balance and daily limit were never tried	1 of 10 points = 10%
Error guessing	Inputs 0 and abc	Useful but unplanned

The testers chose representative values but never derived boundaries or the daily-limit and cash rules. Class coverage is acceptable, boundary coverage is inadequate, and the daily-limit and ATM-cash components are untested.

5. Reconstructed improved test set

Default data: balance Rs. 30,000, daily total used Rs. 0, ATM cash sufficient.

Test	Input	Condition / boundary	Expected result
T01	99	Below minimum (EC3)	Rejected: below minimum
T02	100	Minimum boundary	Success, balance 29,900
T03	200	Just above minimum	Success
T04	19,900	Just below maximum	Success
T05	20,000	Maximum boundary	Success, balance 10,000
T06	20,100	Just above maximum (EC6)	Rejected: per-transaction limit
T07	550	Wrong denomination (EC4)	Rejected: multiples of 100
T08	0 and −100	EC1	Rejected: invalid amount
T09	abc and blank	EC2	Rejected: invalid amount
T10	5,000 (balance 5,000)	Amount = balance	Success, balance 0
T11	5,100 (balance 5,000)	Balance + 100 (EC7)	Rejected: insufficient balance

Test	Input	Condition / boundary	Expected result
T12	5,000 (balance 4,900)	Just below the needed balance	Rejected: insufficient balance
T13	10,000 (daily used 40,000)	Daily total = 50,000	Success
T14	10,100 (daily used 40,000)	Daily total = 50,100 (EC8)	Rejected: daily limit
T15	100 (daily used 50,000)	Limit already reached	Rejected: daily limit
T16	5,100 (ATM cash 5,000)	Cash availability	Rejected: cash not available, no debit
T17	5,000, dispenser jams	Failure after debit	Debit reversed, message shown
T18	Same request sent twice	Duplicate submission	Debited only once

6. Justification

- **Complete partition coverage:** all 8 classes are exercised (100% against 75% before) and all 10 boundary points are tested (100% against 10%).
- **Defect detection:** a wrong relational operator (> instead of >=, < instead of <=) is caught by the adjacent pairs T01/T02, T05/T06 and T10/T11.
- **Hidden components covered:** the daily limit, cash availability and failure atomicity (T13 to T18) were invisible in the original log and are now explicit.

Q2. White-Box Reverse Engineering from Execution Traces and Code

1. Understanding of the system

The module calculates a bill: each price above 1,000 gets a 10% discount, and a member gets a further 5% on the total. No test documentation exists, only the code snippet and two execution traces (both illustrative).

```
i = 0; total = 0;                    // node 1
while (i < n) {                      // node 2
  if (price[i] > 1000) total += 0.9 * price[i];   // nodes 3, 4
  else total += price[i];                         // node 5
  i++;                                            // node 6
}
if (member) total = total * 0.95;        // nodes 7, 8
print total;                         // node 9
```

2. Control flow graph

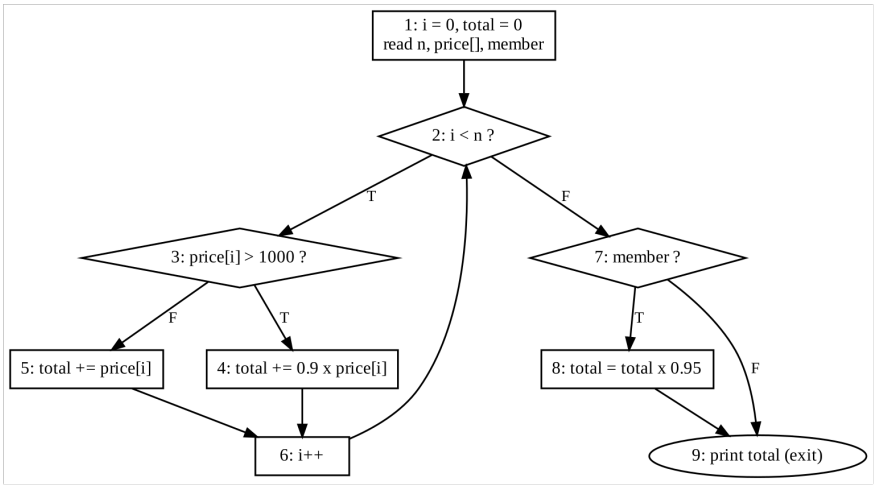


Fig. 2: Control flow graph reconstructed from the code

Nodes N = 9, edges E = 11, predicate nodes P = 3 (nodes 2, 3 and 7). Cyclomatic complexity V(G) = E − N + 2 = 11 − 9 + 2 = **4**, which equals P + 1 = 4.

3. Decision points, loops and independent paths

- **Decision points:** node 2 (loop condition i < n), node 3 (price above 1,000) and node 7 (member).
- **Loop:** nodes 2 to 6, with the back edge 6 → 2 and the zero-iteration exit 2 → 7.

Path	Sequence of nodes	Scenario
BP1	1-2-7-9	No items (loop skipped), not a member
BP2	1-2-3-5-6-2-7-9	One item at or below 1,000, not a member (baseline)
BP3	1-2-3-4-6-2-7-9	One item above 1,000 (discount branch)
BP4	1-2-3-5-6-2-7-8-9	Member discount applied

4. Reverse engineering of the existing tests and their adequacy

Trace	Input	Path executed	Observation
Trace A	n = 2, price = [500, 200], member = false	1-2-3-5-6-2-3-5-6-2-7-9	Only the else branch, loop run twice
Trace B	n = 1, price = [1500], member = false	1-2-3-4-6-2-7-9	Discount branch, loop run once

Both traces are normal "happy-path" runs with a non-member, so the approach was most likely statement-oriented testing from typical data. The adequacy against coverage criteria is:

Criterion	Calculation	Achieved
Statement coverage	8 of 9 nodes (node 8 never executed)	88.9%
Branch (decision) coverage	5 of 6 outcomes (member = true never taken)	83.3%
Basis path coverage	2 of 4 (BP1 and BP4 not run)	50%
Loop coverage	Loop run once and twice, zero iterations never	2 of 3 = 66.7%

5. Reconstructed complete path testing strategy

The criterion is all 4 basis paths, 100% branch coverage and the loop tests (zero, one, two and many iterations) with boundary price values (1,000 and 1,001).

Test	Input	Path / purpose	Expected total
TP1	n = 0, member = false	BP1 (loop skipped)	0.00
TP2	n = 0, member = true	Member branch with empty loop	0.00
TP3	[500], false	BP2	500.00
TP4	[1500], false	BP3	1,350.00
TP5	[500], true	BP4	475.00
TP6	[1000, 1001], false	Boundary of node 3, loop twice	1,900.90
TP7	[1500, 2000], true	Both branches and member	2,992.50
TP8	[100, 2000, 300], false	Loop many times, alternating branches	2,200.00

Criterion	Before	After TP1 to TP8
Statement	88.9%	100% (9 of 9)

Criterion	Before	After TP1 to TP8
Branch	83.3%	100% (6 of 6)
Basis path	50%	100% (4 of 4)
Loop (0, 1, many)	66.7%	100%

The expected totals come from the requirement (10% above 1,000, 5% for members), not from the code, so a wrong operator or factor on any path is detected and not just executed.

Q3. Web Login and Transaction System – Reverse Engineering from Defect Logs

1. Understanding of the system and the available evidence

Only defect logs and test results exist (both illustrative). Assumed rules: user name 1 to 20 characters, password 8 to 64 characters, account locked after 5 wrong passwords, transaction amount 1.00 to 1,00,000.00 with two decimals, amount not above the balance.

Area	Executed	Passed	Failed	Failure rate
Login	40	35	5	12.5%
Transaction	60	55	5	8.3%
Total	100	90	10	10%

ID	Area	Defect	Detected by
D1	Login	Password of exactly 64 characters rejected as too long	Boundary test
D2	Login	Blank user name gives a blank page without a message	Invalid-class test
D3	Login	Account locked after 3 wrong attempts (requirement: 5)	Counter boundary
D4	Login	Input ' OR 1=1 in the user field returns server error 500	Error guessing
D5	Login	Logged-out user opens the transaction URL directly	Exploratory (late)
D6	Transaction	Amount 1,00,001 accepted (maximum is 1,00,000)	Boundary test
D7	Transaction	Amount equal to the balance rejected as insufficient	Boundary test
D8	Transaction	Transfer to the user's own account allowed	Requirement check
D9	Transaction	Session timeout during posting: debit done, no confirmation, retry debits twice	Field report
D10	Transaction	Amount 10.005 (three decimals) accepted and rounded wrongly	Invalid-class test

2. Defect patterns and the techniques most likely used

Pattern in the defects	Defects	Technique inferred
Failures exactly at limits (64 characters, 1,00,001, amount = balance, 5th/3rd failure)	D1, D3, D6, D7	Black box: boundary value analysis
Failures for invalid inputs (blank, decimals, SQL characters)	D2, D4, D10	Black box: equivalence partitioning of invalid classes and error guessing
Failures only on unusual flows (direct URL, timeout mid-posting)	D5, D9	Found late or in the field: no structural path or state testing was used
Business rule broken only in one combination	D8	Functional test of one rule, no decision table

No white-box technique can be identified from the logs; the pattern in D5 (an authorization branch never executed) indicates that, at best, statement coverage was measured and branch coverage was not.

3. Decomposition into testable components

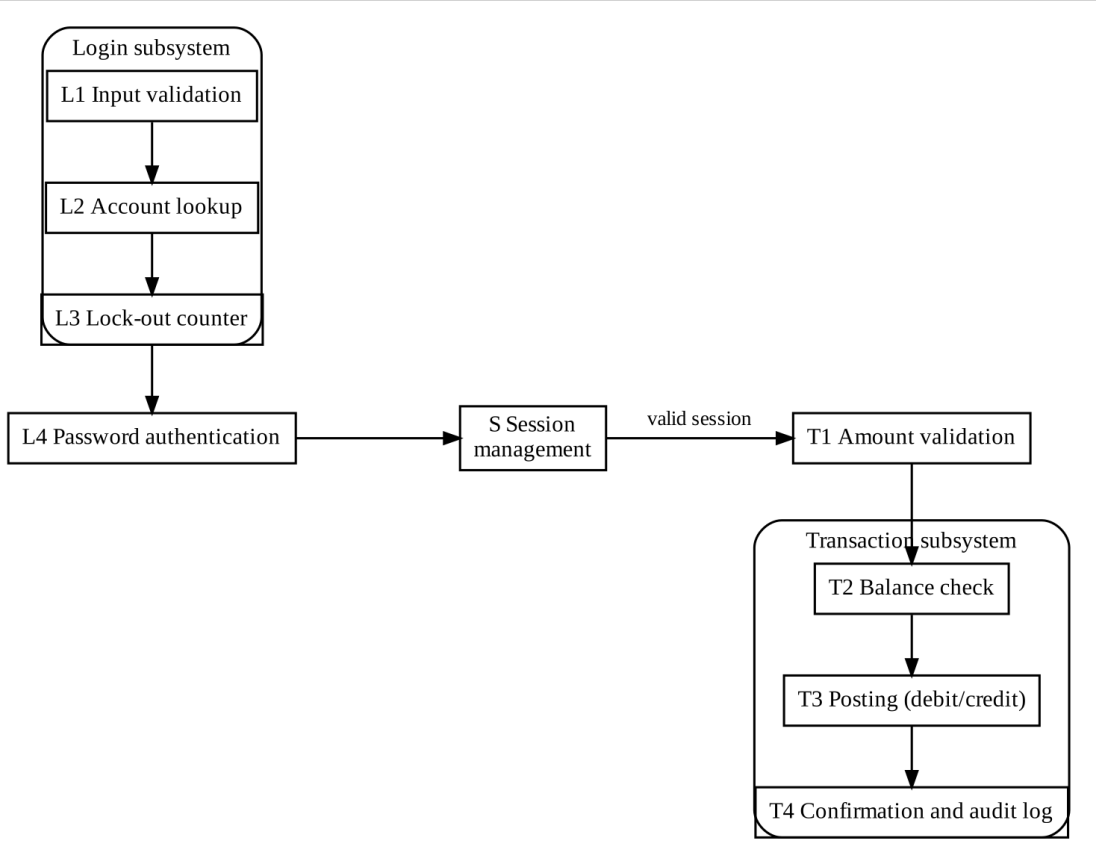


Fig. 3: Testable components of the login and transaction system

4. Gaps in the testing coverage

Gap	Evidence	Missing technique
Session and authorization paths	D5, D9	Path/branch testing, state-based session tests
Rule combinations (own account, balance, amount)	D8	Decision table testing
Numeric precision of amounts	D10	Equivalence class for decimals
Lock-out counter at 4, 5 and 6 failures	D3	Systematic boundary values
Retry, double submit, timeout	D9	Stress and idempotency tests

5. Reconstructed comprehensive testing strategy

- **Equivalence partitioning (15 tests):** user name (valid + blank, over 20, unknown, special characters), password (valid + blank, under 8, over 64, wrong) and amount (valid + zero or negative, over 1,00,000, more than two decimals, non-numeric): 1 valid and 4 invalid classes per field.
- **Boundary value analysis (17 tests):** password length 7, 8, 9, 63, 64, 65; amount 0.99, 1.00, 1.01, 99,999.99, 1,00,000.00, 1,00,000.01; amount = balance and balance + 0.01; wrong attempts 4, 5 and the 6th attempt with the correct password.
- **Path testing (10 tests):** the login logic has 5 decisions (blank fields, unknown user, locked account, password correct, attempts at the limit), so V(G) = 6 basis paths: blank, unknown user, locked, success, wrong below the limit, wrong reaching the limit. The transaction module has 3 decisions (session valid, amount valid, balance

enough), so V(G) = 4 paths.

- **Decision table (5 tests):** the 4 conditions (session valid, amount valid, balance sufficient, target different from source) give 16 combinations that collapse to 5 rules: no session, invalid amount, insufficient balance, same account, success.
- **State, stress and error guessing (3 tests):** session expiry during posting, double click on Submit, network drop after the debit.

Technique	Tests	Defects it would have exposed or prevented
Equivalence partitioning	15	D2, D4, D10
Boundary value analysis	17	D1, D3, D6, D7
Path testing	10	D3, D5
Decision table	5	D5, D8
State / stress / error guessing	3	D4, D9
Total	50	All 10 defects traceable

6. Justification

Every defect type found in the logs maps to at least one designed test, and the gaps (session paths, combinations, retries) are closed by techniques that the original approach lacked. The 50 designed tests give measurable coverage of classes, boundaries, paths and rules, so the result is traceable and repeatable, whereas the 100 earlier tests had no coverage model.

Q4. Legacy Inventory System – Reverse Engineering the Business Logic from Logs

1. Understanding of the system

Only transaction logs of stock updates and low-stock alerts are available (illustrative values).

Log	Operation	Qty	Stock before	Result	Stock after	Alert
L1	ISSUE	20	100	OK	80	none
L2	ISSUE	60	80	OK	20	LOW-STOCK
L3	ISSUE	20	20	OK	0	LOW-STOCK
L4	ISSUE	70	100	OK	30	none
L5	RECEIVE	500	50	OK	550	none
L6	ISSUE	21	20	FAIL: insufficient stock	20	none
L7	ISSUE	0	50	FAIL: invalid quantity	50	none
L8	ISSUE	−5	80	FAIL: invalid quantity	80	none
L9	RECEIVE	200	900	FAIL: exceeds capacity	900	none

2. Hidden business rules inferred from the logs

Rule	Inference	Evidence	Remaining uncertainty
H1	Quantity must be a positive number	L7, L8	Is a decimal quantity allowed?
H2	Issue allowed if qty ≤ stock	L3 (equal is accepted), L6 (stock + 1 is rejected)	None: boundary pinned on both sides
H3	Receive allowed if stock + qty ≤ capacity C	L5 (550 accepted), L9 (1,100 rejected)	550 ≤ C < 1,100; probably 1,000
H4	Alert when stock after update ≤ reorder level R	L2, L3 alert at 20 and 0; L4 none at 30	20 ≤ R < 30; probably 20
H5	A failed transaction leaves stock unchanged	L6 to L9	None

3. Decomposition into functional units and equivalence partitions

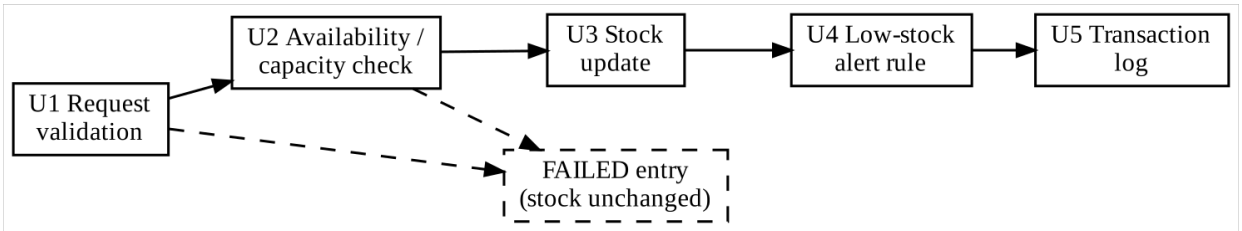


Fig. 4: Functional units of the stock-update process

Unit	Partitions (valid / invalid)
U1 Request validation	qty ≥ 1 integer (valid); qty ≤ 0, decimal, non-numeric, unknown item (invalid)
U2 Availability / capacity	ISSUE: 1 ≤ qty ≤ stock (valid), qty > stock (invalid); RECEIVE: stock + qty ≤ C (valid), above C (invalid)
U3 Stock update	new stock = stock qty, never negative, never above C
U4 Alert rule	stock after update > R (no alert); 0 ≤ stock ≤ R (alert); alert raised once, cleared when stock rises above R
U5 Transaction log	one entry for every success and every failure

4. Gaps found in the logs

- **Quantity 1** (the minimum) and decimal quantities were never tried.
- **Capacity boundary:** receiving up to exactly C and C + 1 is missing; C itself is unknown.
- **Alert threshold:** stock values 21 to 29 are untested, so R is not pinned down; alert repetition and alert clearing are not covered.
- **Other:** unknown item and two simultaneous updates of the same item.

5. Reconstructed black-box test design

Working values: C = 1,000 and R = 20 (to be confirmed with the business owner; tests B10 and B11 locate R if the hypothesis is wrong).

Test	Stock before	Operation	Qty	Expected result	Gap closed
B1	100	ISSUE	1	OK, stock 99, no alert	Minimum quantity
B2	100	ISSUE	0	FAIL: invalid quantity	H1

Test	Stock before	Operation	Qty	Expected result	Gap closed
B3	100	ISSUE	−1	FAIL: invalid quantity	H1
B4	100	ISSUE	1.5	FAIL: invalid quantity	Decimal quantity
B5	100	ISSUE	100	OK, stock 0, alert	H2 boundary
B6	100	ISSUE	101	FAIL: insufficient stock	H2 boundary
B7	100	ISSUE	79	OK, stock 21, no alert	R + 1
B8	100	ISSUE	80	OK, stock 20, alert	R
B9	100	ISSUE	81	OK, stock 19, alert	R − 1
B10	100	ISSUE	71	OK, stock 29, no alert	Upper probe for R
B11	100	ISSUE	75	OK, stock 25, result recorded	Locates R
B12	900	RECEIVE	100	OK, stock 1,000, no alert	C boundary
B13	900	RECEIVE	101	FAIL: exceeds capacity	C + 1
B14	15 (alert active)	ISSUE	1	OK, stock 14, no second alert	Alert repetition
B15	15 (alert active)	RECEIVE	100	OK, stock 115, alert cleared	Alert clearing
B16	any	ISSUE	1 (unknown item)	FAIL: item not found	U1
B17	100	2 x ISSUE at once	60 each	One OK, one FAIL, stock never negative	Concurrency

6. Justification of the selection

Each test is derived from an inferred rule: H2 by B5 and B6, H3 by B12 and B13, H4 by B7 to B11, H1 by B1 to B4, and H5 by the unchanged stock after every FAIL. Boundary pairs (B5/B6, B12/B13, B7/B8/B9) detect a wrong operator or a threshold off by one, while B14 to B17 cover state and concurrency behaviour that single-input logs cannot show. The 17 tests cover all partitions of units U1 to U5.

Q5. Library Management .exe – White-Box Requirements from a CPU/Memory Profile

1. Understanding of the system

Only a compiled library-management module and its CPU/memory profile during a stress test are available. The profile below is illustrative; the module handles search, issue, return and fine requests.

Phase	Time (s)	CPU (%)	Memory (MB)	Observation
P0	0 to 10	5	40	Start-up, idle
P1	10 to 25	35	40 to 45	Steady load
P2	25 to 40	95	45 to 480	CPU spike with linear memory growth
P3	40 to 50	100	480 (flat)	CPU saturated, no memory growth
P4	50 to 60	8	480 (flat)	Load stopped, memory not released

2. Mapping execution spikes to loops and decision clusters

Phase	Probable logic	CFG nodes	Reasoning
P1	Request loop with simple branching on request type	2, 3, 4, 8, 11 to 14	CPU grows with load only
P2	Catalogue scan loop nested in the request loop, results accumulated	5, 6, 7	Requests × books iterations; memory grows with every match added
P3	Lock retry loop with no wait or back-off	9, 10	100% CPU with flat memory means a spin loop that allocates nothing
P4	Objects kept after the request returns to the loop head	edges 5 → 2, 12 → 2, 13 → 2	Memory is not released: probable leak

3. Conceptual control flow graph

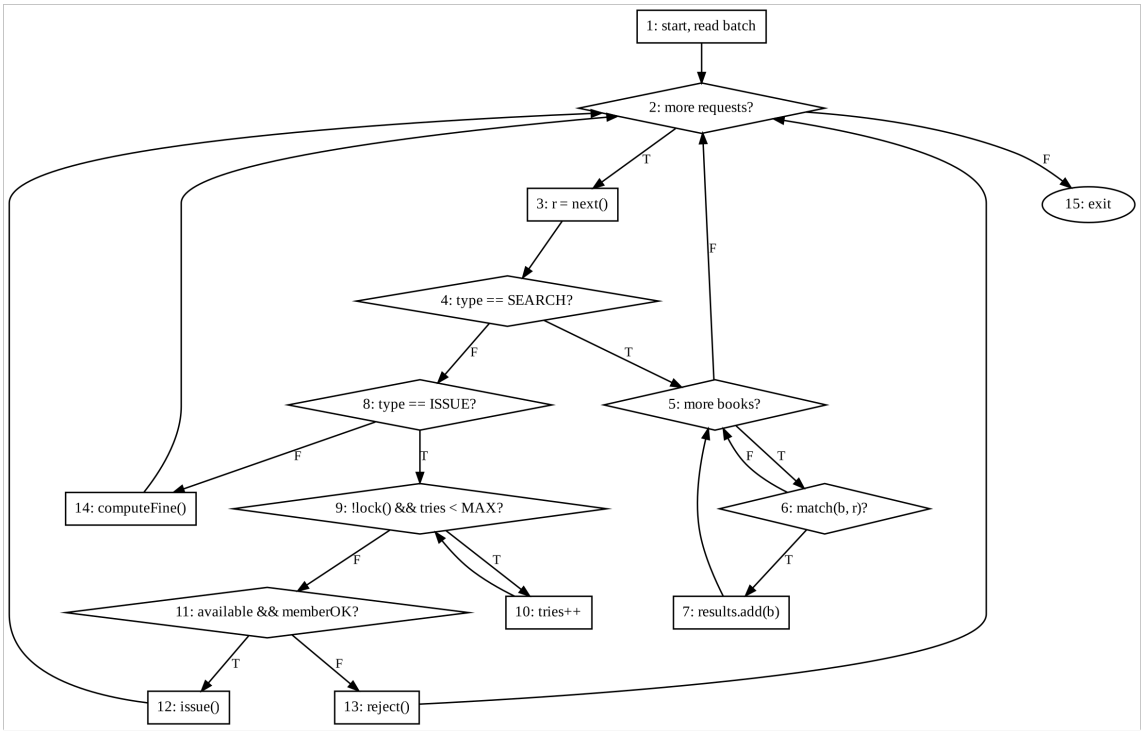


Fig. 5: Conceptual CFG of the library module reconstructed from the profile

Nodes N = 15, edges E = 21, predicate nodes P = 7 (nodes 2, 4, 5, 6, 8, 9, 11). V(G) = E − N + 2 = 21 − 15 + 2 = **8**, which equals P + 1. Eight independent paths are required.

Path	Sequence of nodes	Scenario	Resource risk
BP1	1-2-15	No request	Low
BP2	1-2-3-4-5-2-15	Search, empty catalogue	Low
BP3	1-2-3-4-5-6-5-2-15	Search, no match	CPU: full scan
BP4	1-2-3-4-5-6-7-5-2-15	Search, match added	CPU and memory (P2)
BP5	1-2-3-4-8-9-11-12-2-15	Issue, lock free	Low
BP6	1-2-3-4-8-9-10-9-11-12-2-15	Issue after a lock retry	CPU spin (P3)
BP7	1-2-3-4-8-9-11-13-2-15	Issue rejected	Memory if not released (P4)

Path	Sequence of nodes	Scenario	Resource risk
BP8	1-2-3-4-8-14-2-15	Fine calculation	Low

4. Path testing strategy for the resource-intensive paths

Test	Path	Loop settings	Acceptance criterion
RP1	BP4	Catalogue 10, 1,000, 100,000 books; every book matches	CPU time grows linearly, results are paged and capped
RP2	BP6	tries = 0, 1, MAX − 1, MAX	Retry has a back-off; no CPU spin; failure message after MAX
RP3	BP5, BP7	10^6 requests in a row	Memory returns to ≤ 1.2 × the 40 MB baseline after the run
RP4	BP3, BP4	Outer loop at its maximum and inner loop at its maximum (nested-loop rule)	No overflow, no timeout
RP5	BP1, BP2, BP8	Zero and one iteration	Correct result, no allocation left
RP6	All	Concurrent users 1, 10, 100, 1,000	Throughput degrades gradually, no crash

5. Impact on system stability

Metric	Observed in the profile	Target after the path-based fixes
Peak CPU	100% for 10 s (spin loop)	Below 70% sustained
Memory after the load	480 MB, not released	≤ 1.2 × baseline (48 MB)
Memory growth during search	Linear and unbounded	Bounded by the page size
Lock wait	Busy waiting	Waiting with back-off

The targets are acceptance criteria for the reconstructed tests, not measured results. Testing the eight basis paths with the loop boundaries makes each spike reproducible and gives a pass/fail decision for stability.

Q6. Mobile Payment Gateway – Reverse Engineering the State Machine

1. Understanding of the system

Sample JSON responses (illustrative) for one transaction lifecycle:

{"txn_id":"T1001","status":"PENDING","amount":500.00,"expires_in":300}
{"txn_id":"T1001","status":"SUCCESS","amount":500.00,"auth_code":"A7X2K9"}
{"txn_id":"T1001","status":"REVERSAL","refund_amount":500.00,"reason":"CUSTOMER_REQUEST"}
{"txn_id":"T1002","status":"TIMEOUT","error_code":"E408","retry_allowed":true,"retries":1}

Response	Key fields	Data constraint inferred
PENDING	amount, expires_in = 300	amount > 0; expires after 300 s
SUCCESS	auth_code present	Same amount as the PENDING response; auth_code only here
REVERSAL	refund_amount, reason	refund_amount equals amount; only after SUCCESS or a debited TIMEOUT
TIMEOUT	error_code E408, retry_allowed, retries	retries ≤ 3; after the third retry retry_allowed = false

2. Reverse-engineered state machine

States: S0 Initiated (implicit, before the first response), S1 Pending, S2 Success, S3 Reversal (final) and S4 Timeout. Events: create, bank_confirm, timer_expire, refund_request, retry and auto_reverse.

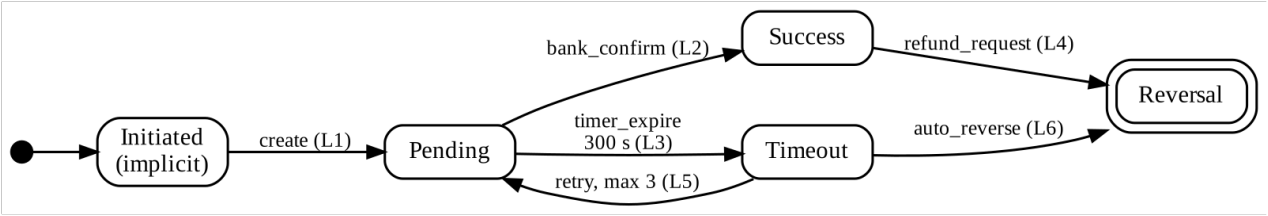


Fig. 6: State-transition diagram of the payment gateway

Id	Transition	Trigger and constraint
L1	Initiated → Pending	create with amount > 0
L2	Pending → Success	bank_confirm within 300 s, auth_code issued
L3	Pending → Timeout	no bank response after 300 s
L4	Success → Reversal	refund_request, refund_amount = amount
L5	Timeout → Pending	retry while retries ≤ 3 and retry_allowed = true
L6	Timeout → Reversal	auto_reverse when the amount was debited

3. Implicit technique identified

The gateway was designed for **state transition testing**: the response depends on the current state and the event, and the data constraints (300 s, 3 retries, refund = amount) act as guards. The suitable coverage measures are all-states, all-transitions (0-switch) and 1-switch sequences.

4. State table (valid and illegal transitions)

State	create	bank_confirm	timer_expire	refund_request	retry	auto_reverse
Initiated	Pending	–	–	–	–	–
Pending	–	Success	Timeout	–	–	–
Success	–	–	–	Reversal	–	–
Reversal	–	–	–	–	–	–
Timeout	–	–	–	–	Pending	Reversal

The table has 5 × 6 = 30 cells: 6 legal transitions and 24 illegal ones (shown as –).

5. Reconstructed state-based test suite

Test	Event sequence	Expected final state
TS1	create, bank_confirm	Success (L1, L2)
TS2	create, timer_expire	Timeout (L1, L3)
TS3	create, bank_confirm, refund_request	Reversal (L1, L2, L4)
TS4	create, timer_expire, retry, bank_confirm	Success (L1, L3, L5, L2)
TS5	create, timer_expire, auto_reverse	Reversal (L1, L3, L6)
TS6	create, then expire and retry three times, then retry again	Timeout; fourth retry rejected (L1, L3, L5, L3, L5, L3, L5, L3)

These six sequences cover all 5 states, all 6 legal transitions and all 7 legal 1-switch pairs (L1-L2, L1-L3, L2-L4, L3-L5, L3-L6, L5-L2, L5-L3).

Tests	State	Illegal events (each must be rejected, state unchanged)
ST07 to ST11	Initiated	bank_confirm, timer_expire, refund_request, retry, auto_reverse
ST12 to ST15	Pending	create, refund_request, retry, auto_reverse
ST16 to ST20	Success	create, bank_confirm, timer_expire, retry, auto_reverse
ST21 to ST26	Reversal	all six events (including a second refund_request)
ST27 to ST30	Timeout	create, bank_confirm, timer_expire, refund_request

Test	Data-constraint check	Expected result
DC1	bank_confirm at 299 s	Success
DC2	bank_confirm at 300 s and 301 s	Timeout (late confirmation not accepted)
DC3	refund_amount = amount + 0.01	Rejected
DC4	create with amount = 0	Rejected, no state created

The suite has 6 + 24 + 4 = 34 tests; every legal and every illegal transition is exercised and the guards are tested at their boundaries.

Q7. Encrypted Communication Module – Fuzzing and Boundary Value Analysis

1. Understanding of the system

The module offers encrypt(message, key, recipientId) and returns a Base64 ciphertext or an error code. The source is not available, so the validation logic is reconstructed by fuzzing the input fields. The results below are illustrative.

Fuzz	Input	Observation	Inference
F1	message of 0 characters	E01 empty message	Minimum length 1
F2	message of 1 and of 1,024 characters	OK	Both lengths accepted
F3	message of 1,025 and 65,536 characters	E02 too long, no crash	Maximum length 1,024, checked early
F4	1,024 four-byte characters (4,096 bytes)	OK	Limit counts characters, not bytes: buffer risk
F5	NUL character inside the message	OK, but the decrypted text stops at the NUL	NUL handled as a string terminator (defect)
F6	key of 15, 16, 17, 24, 32, 33 bytes	OK only for 16, 24 and 32	Key length must be 16, 24 or 32
F7	recipientId of 0, 32, 33 characters	E04 for 0 and 33	Length 1 to 32
F8	recipientId with quotes, ../ or %s%n	E04	Allow-list of characters
F9	missing parameter, number instead of text	E05, E06	Presence and type are checked
F10	same input twice	Different ciphertexts	Random IV for every call

2. Decomposition into validation and encryption components

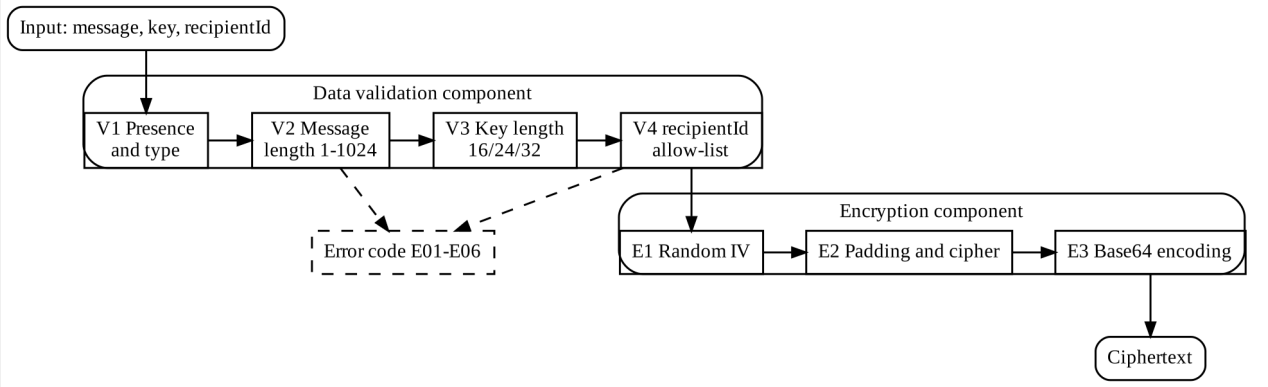


Fig. 7: Decomposition of the module inferred from the fuzzing

The inferred order is V1 presence and type (E05, E06), V2 message length 1 to 1,024 (E01, E02), V3 key length (E03), V4 recipientId pattern [A-Za-z0-9_-]{1,32} (E04), followed by the encryption component (random IV, padding and cipher, Base64 encoding).

3. Reconstructed test cases using Boundary Value Analysis

Input	Boundary values	Expected result
Message length (characters)	0, 1, 2, 1,023, 1,024, 1,025	0: E01; 1 to 1,024: OK with correct round trip; 1,025: E02 before any copy or encryption
Message size (bytes)	1,024 characters of four bytes each	Limit enforced in bytes: rejected or buffer sized for 4 × 1,024 bytes
Extreme sizes	65,536 characters, 1 MB, $2^{31} - 1$	E02 quickly, no crash, no memory growth
Cipher block boundary (bytes)	15, 16, 17, 31, 32, 33	decrypt(encrypt(m)) = m (padding boundary)
Key length (bytes)	15, 16, 17, 23, 24, 25, 31, 32, 33	Only 16, 24, 32 accepted; others E03
recipientId length	0, 1, 2, 31, 32, 33	0 and 33: E04; others OK
Special characters	NUL, CR/LF, quote, semicolon, ../, %s%n, angle brackets	In recipientId: E04. In message: processed as data, round trip exact, no log forging
Fuzz regression	10,000 seeded random inputs	No crash and no memory-checker error; every reject returns a defined error code

4. How the design prevents buffer overflow and injection

- **Buffer overflow:** every input has an explicit maximum, and the tests at maximum + 1 and at extreme sizes show that the validator rejects the value before any buffer copy. The byte-versus-character test and the NUL test expose length and terminator errors, the two usual causes of overflows in string handling.
- **Injection:** recipientId is accepted only through an allow-list, so quotes, path characters and format strings are rejected (E04). The message is treated only as data and is never joined into a query, command or log line, which the CR/LF and quote tests verify.
- **Encryption correctness:** the block-boundary round trips, the random IV check and the wrong-key decryption failure confirm that validation did not weaken the cipher.
- **Fail-closed behaviour:** every invalid input gives a fixed error code and no partial ciphertext.

Q8. Insurance Premium Calculator – 100% Statement Coverage but Missed Defects

1. Understanding of the system and the original approach

The calculator (illustrative code) applies loadings for young drivers with accidents, for smokers or high BMI, and a cap:

```
premium = base;                          // node 1
if (age < 25 && accidents > 0) premium *= 1.5;    // nodes 2, 3, 4
if (smoker || bmi > 30)        premium *= 1.3;    // nodes 5, 6, 7
if (premium > cap)             premium = cap;     // nodes 8, 9
return premium;                          // node 10
```

One test (age 20, accidents 1, smoker, bmi 22, base 1,200) executes every statement, so statement coverage is 100%. This shows that the original white-box approach used **statement coverage only**.

2. Inadequacy of the coverage criterion

Criterion	With the single legacy test	Gap
Statement	Every statement executed = 100%	Satisfied without ever taking a false outcome
Branch (decision outcomes)	3 of 6 = 50%	The false outcome of all three ifs is never taken
Condition	4 of 10 outcomes = 40%	No condition is ever false, and bmi is never evaluated because smoker is true
Path	1 of 8 = 12.5%	Seven combinations never run

A fault such as && written as || in the first if, or smoker || bmi written as smoker && bmi, gives the same answer when all conditions are true, so it stays hidden.

3. New control flow graph and independent paths

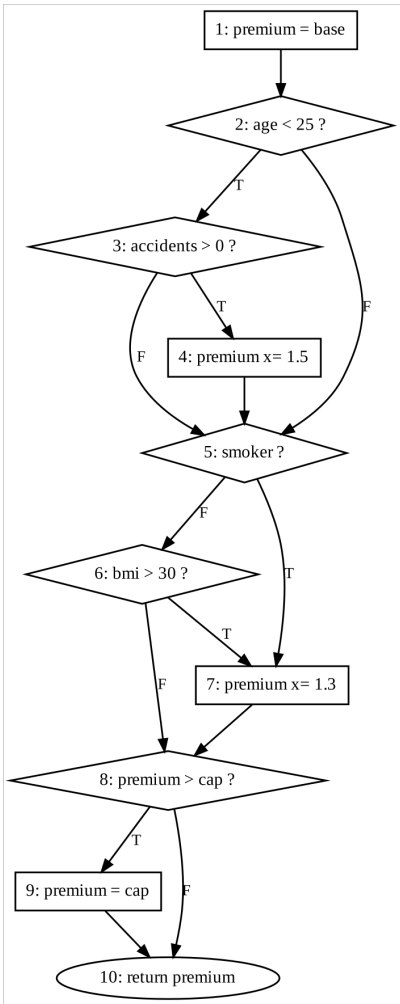


Fig. 8: CFG at condition level for the premium calculator

N = 10, E = 14, atomic predicates = 5 (nodes 2, 3, 5, 6, 8). V(G) = 14 − 10 + 2 = **6**, which equals 5 + 1. At decision level the three ifs are independent and in sequence, so there are 2 × 2 × 2 = **8** complete paths.

Path	**Nodes**	**Decisions (D1 young with accident, D2 smoker or BMI, D3 cap)**
P1	1-2-5-6-8-10	D1 false, D2 false, D3 false
P2	1-2-5-6-8-9-10	D1 false, D2 false, D3 true
P3	1-2-5-7-8-10	D1 false, D2 true, D3 false
P4	1-2-5-7-8-9-10	D1 false, D2 true, D3 true
P5	1-2-3-4-5-6-8-10	D1 true, D2 false, D3 false
P6	1-2-3-4-5-6-8-9-10	D1 true, D2 false, D3 true
P7	1-2-3-4-5-7-8-10	D1 true, D2 true, D3 false
P8	1-2-3-4-5-7-8-9-10	D1 true, D2 true, D3 true

4. Reconstructed path testing strategy (cap = 2,000)

Test	age	accidents	smoker	bmi	base	Path	Expected premium
T1	30	0	No	22	1,000	P1	1,000
T2	30	0	No	22	2,500	P2	2,000
T3	30	0	Yes	22	1,000	P3	1,300
T4	30	0	Yes	22	2,000	P4	2,000
T5	20	1	No	22	1,000	P5	1,500
T6	20	1	No	22	1,500	P6	2,000
T7	20	1	Yes	22	1,000	P7	1,950
T8	20	1	Yes	22	1,200	P8	2,000

T1 to T8 give 100% statement, 100% branch and 100% path coverage. Condition-level and boundary tests add the individual conditions:

Test	Input	Purpose	Expected premium
T9	age 20, accidents 0, base 1,000	Only age true: detects && replaced by ||	1,000
T10	age 30, accidents 1, base 1,000	Only accidents true	1,000
T11	non-smoker, bmi 35, base 1,000	Only bmi true: detects || replaced by &&	1,300
T12 / T13	age 24 / 25, accidents 1, base 1,000	Boundary of age < 25	1,500 / 1,000
T14 / T15	non-smoker, bmi 30 / 31, base 1,000	Boundary of bmi > 30	1,000 / 1,300
T16	no loadings, base 2,000	premium equal to cap is unchanged	2,000

5. How the reconstruction reduces the risk of calculation errors

- **Both outcomes of every decision and every atomic condition are tested, so operator errors (&& / ||, < / <=) change the result and are detected.**
- **Path P7 (both loadings) gives 1,950, where an additive error would give 1,800; combination effects of the loadings are therefore checked.**
- **The cap is tested below, above and exactly at its value (T1 to T8, T16), so wrong cap handling cannot cause an overcharge.**

Q9. Cloud File Synchronization – Reverse Engineering from "Lost File" Defects

1. Understanding of the system

The tool detects changed files, filters them, splits them into 4 MB chunks, uploads them over the network, verifies them on the server and marks them as synced. The activity log shows successful syncs of 0.5 MB, 5 MB, 9 MB, 20 MB and 500 MB files, all at 20 Mbps. The "lost file" reports are illustrative:

ID	Size	Type	Network	Outcome
LF1	4 MB	pdf	20 Mbps	Lost
LF2	8 MB	jpg	50 Mbps	Lost
LF3	100 MB	zip	20 Mbps	Lost
LF4	0 bytes	txt	20 Mbps	Never uploaded
LF5	600 MB	mp4	0.5 Mbps	Lost, marked as synced

ID	Size	Type	Network	Outcome
LF6	300 MB	mp4	1 Mbps	Lost
LF7	12 KB	~$report.docx	20 Mbps	Skipped silently
LF8	2 KB	tmp	20 Mbps	Skipped silently
LF9	40 MB	docx	drops at 60%	Lost, no resume
LF10	30 KB	file name of 260 characters	20 Mbps	Lost

2. Decomposition and defect patterns

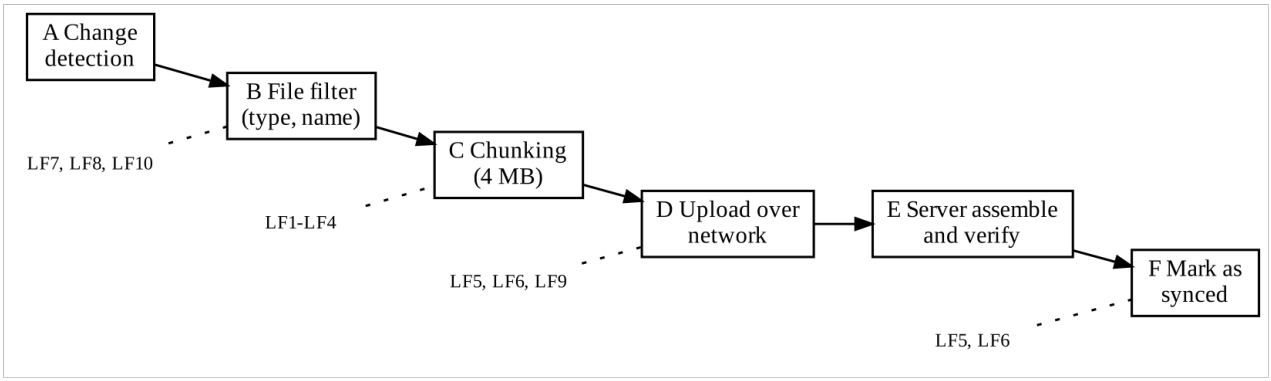


Fig. 9: Synchronization pipeline and the stage where each defect arises

Pattern	Reports	Inferred cause
Exact multiples of the 4 MB chunk (4, 8, 100 MB) are lost, while 5 MB and 9 MB sync	LF1 to LF3	Off-by-one in the handling of the last chunk
Empty file	LF4	Size 0 not handled as a valid class
Large file on a slow network	LF5, LF6	Timeout treated as success and the file marked as synced
Temporary and long-named files	LF7, LF8, LF10	Filter and name-length rules without a user message
Connection drop during upload	LF9	No resume or retry

3. Inferred equivalence class partitioning of the developers

Input	Likely classes used	Values the logs suggest were tested
File size	Empty, small (below 1 MB), medium (1 to 100 MB), large (above 100 MB)	0.5, 5, 9, 20 and 500 MB (mid-class values only)
Network speed	Fast (10 Mbps or more), medium (1 to 10), slow (below 1)	20 Mbps only; slow class only with small files
File type	Documents, images, media, archives	Common types; temporary and system files not tried

The defects cluster at class edges (4 MB multiples, 0 bytes, slow network with large files) and in conditions that need stress (interruption), so boundary values, slow-network combinations and stress were missing.

4. Revised testing strategy: BVA with stress testing

Test group	Values / conditions	Expected result
Size BVA (chunk 4 MB)	0 B, 1 B, 4 MB − 1 B, 4 MB, 4 MB + 1 B, 8 MB, 100 MB − 1 B, 100 MB, 100 MB + 1 B	Every file synced; SHA-256 of the server copy equals the source
Maximum size (assumed 5 GB)	5 GB − 1 B, 5 GB, 5 GB + 1 B	Accepted, accepted, rejected with a visible message
Network BVA	Offline, 0.1, 0.5, 1.0, 1.01, 10 Mbps with a 100 MB file	Completes or queues; never marked synced before verification
Class combination	500 MB at 0.5 Mbps; 0 B at 0.5 Mbps	Correct result
File names	254, 255, 256 characters; special characters	255 accepted; others rejected with a message
Skipped files	~$ and .tmp files	Skipped only by rule and shown in a log or notice
Stress	1,000 small files together; 10 files of 1 GB in parallel; disk 95% full	No file lost, queue completes
Interruption	Drop at 0%, 50%, 99%; five drops in a row; file changed during upload	Resume from the last verified chunk; newest version kept
Soak	24 hours with 10,000 files	No lost file, no memory growth

5. How the reconstructed approach prevents data loss

- **Verification oracle:** a file is marked as synced only when the server checksum equals the source checksum, so timeout-as-success (LF5, LF6) cannot happen.
- **Boundary tests** at every chunk multiple and class edge remove the off-by-one defects (LF1 to LF4).
- **No silent skips:** each skipped file must produce a notice (LF7, LF8, LF10).
- **Stress and interruption tests** force resume, retry and conflict handling before release (LF9), and the automated regression suite keeps the checks in place.

Q10. E-commerce Search and Filter – Decision Table Reconstruction from SQL Queries

1. Understanding of the system

Only generated SQL queries and the returned records are available. The sample dataset (illustrative) and the observed queries are:

ID	Category	Brand	Price	Rating	In stock
P1	Shoes	Nike	1,000	4.5	Yes
P2	Shoes	Adidas	3,000	4.0	Yes
P3	Shoes	Puma	2,000	3.5	No
P4	Sandals	Nike	800	4.2	Yes
P5	Shoes	Nike	2,800	4.8	No
P6	Sandals	Puma	1,500	3.9	Yes
P7	Shoes	Adidas	500	4.1	Yes
P8	Bags	Nike	2,500	4.6	Yes
P9	Shoes	Nike	4,000	4.4	Yes

Query	WHERE clause	Returned
Q-a	category = 'Shoes' AND price BETWEEN 1000 AND 3000	P1, P2, P3, P5
Q-b	category = 'Shoes' AND brand IN ('Nike','Puma')	P1, P3, P5, P9
Q-c	brand = 'Nike' AND stock > 0 AND price BETWEEN 1000 AND 3000	P1, P8
Q-d	rating >= 4 AND NOT (brand = 'Adidas')	P1, P4, P5, P8, P9
Q-e	category = 'Shoes' AND stock > 0 ORDER BY price	P7, P1, P2, P9

2. Inferred decision logic

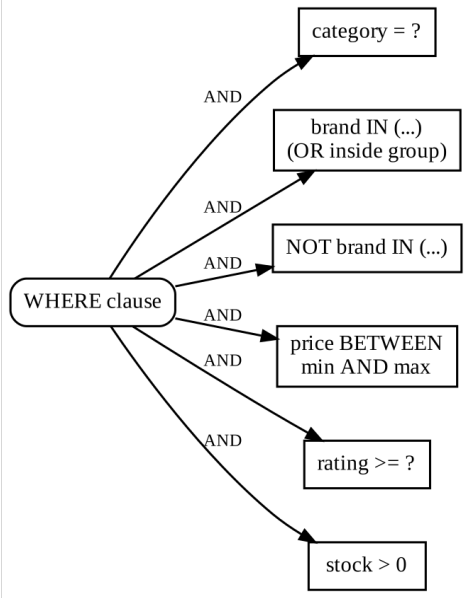


Fig. 10: Reconstructed filter logic: AND between filters, OR inside the brand list

- **Between filters:** AND. A product must satisfy every selected filter, and an unselected filter adds no condition.
- **Inside the brand filter:** OR (IN list), as Q-b returns both Nike and Puma.
- **Exclusion:** NOT brand removes P2 and P7 in Q-d.
- **Price:** inclusive limits (Q-a returns P1 at 1,000 and P2 at 3,000). The rating limit is not visible: P2 (4.0) is the only product at 4.0 and it is removed by the exclusion, so whether the rule is >= or > is a hidden boundary.

3. Reconstructed decision table (core: category, brand, price, in stock)

Test values: category Shoes, brand Nike, price 1,000 to 3,000, in stock only. The four filters give 2 × 2 × 2 × 2 = 16 rules; Y means the filter is selected.

Rule	Cat	Brand	Price	Stock	Expected products
R1	N	N	N	N	P1 to P9
R2	N	N	N	Y	P1, P2, P4, P6, P7, P8, P9
R3	N	N	Y	N	P1, P2, P3, P5, P6, P8
R4	N	N	Y	Y	P1, P2, P6, P8
R5	N	Y	N	N	P1, P4, P5, P8, P9
R6	N	Y	N	Y	P1, P4, P8, P9
R7	N	Y	Y	N	P1, P5, P8

Rule	Cat	Brand	Price	Stock	Expected products
R8	N	Y	Y	Y	P1, P8
R9	Y	N	N	N	P1, P2, P3, P5, P7, P9
R10	Y	N	N	Y	P1, P2, P7, P9
R11	Y	N	Y	N	P1, P2, P3, P5
R12	Y	N	Y	Y	P1, P2
R13	Y	Y	N	N	P1, P5, P9
R14	Y	Y	N	Y	P1, P9
R15	Y	Y	Y	N	P1, P5
R16	Y	Y	Y	Y	P1

Every rule has a different expected result with this dataset, so a defect in any one clause changes at least one result.

4. Extension rules (rating, exclusion, operators and boundaries)

Rule	Condition	Expected result
X1	rating >= 4 only	P1, P2, P4, P5, P7, P8, P9
X2	exclude brand Adidas only	P1, P3, P4, P5, P6, P8, P9
X3	all four core filters + rating >= 4 + exclude Adidas	P1
X4	rating >= 4 + exclude Adidas	P1, P4, P5, P8, P9
X5	category Shoes + brand IN (Nike, Puma)	P1, P3, P5, P9
X6	include Nike and exclude Nike	No products found message
X7	price minimum 3,000 and maximum 1,000	Validation error, no query run
X8	price 1,001 to 2,999 (inclusive limits)	P3, P5, P6, P8 (P1 and P2 excluded)
X9	rating >= 4.1 (hidden boundary)	P1, P4, P5, P7, P8, P9 (P2 at 4.0 excluded)

5. Evaluation of the completeness of the model

Measure	Result
Binary combinations of the 6 filters	$2^6 = 64$
Combinations tested	20 (16 core rules + X1 to X4) = 31%
Pairwise coverage of the 6 filters	60 of 60 filter-state pairs covered = 100%
Single filter on, and each core filter off from all on	Covered (R2, R3, R5, R9, X1, X2; R8, R12, R14, R15)
Operator and boundary rules	X5 to X9 (OR, NOT, conflict, invalid range, inclusive limits)
Total tests	25 instead of 64 or more

The model is complete for the inferred logic, because the filters are combined by AND and each one is independent. The remaining risk is a three-way interaction that involves rating or exclusion (for example a discount-price rule), the empty-search text field and sorting. These are covered by adding pairwise or three-way tests if the SQL later shows such interactions.